



Yes. This file is violating SRP in a major way. The interesting thing is that it is **not just one giant class**—it is effectively an entire application architecture collapsed into one Python module.

I did a structural pass over the complete file. It is **8,651 lines**, with **281 top-level functions**, **97 import statements**, **201 top-level assignments**, **143 uppercase configuration/constants**, and **only 2 actual classes**. The code has already started moving in the right direction: scanner/printer logic has been extracted into `hardware/*`, and the ~1,380-line HTTP handler was moved into `http_app`.

Pasted text(20260824-094016) +1

How many responsibilities?

Using “responsibility” in the SRP sense—a **distinct reason this file might need to change**—I count about **53 responsibilities**, grouped into 14 larger bounded contexts:

Area	Change axes	Examples currently in <code>server.py</code>
Bootstrap/configuration	3	environment, code-change status, startup tasks
Finance reports	5	catalog, parsing, status, resolution, rendering
Recent intake	5	persistence, correlation, reduction, status, halt
Expense/category operations	6	manual entry, edit, taxonomy, recategorize, undo, vendor review
Receipts/supporting docs	4	indexing, matching, source resolution, annotation
Scanner subsystem	5	registry, invocation, locking, diagnostics, staging

Area	Change axes	Examples currently in <code>server.py</code>
Mazda/Trainer	4	execution mode, dispatch, escalation, recovery
Statement/manual-fill	3	preflight, breakup/fill, storage
Agent configuration	3	registry/cards, model selection, voice
Letta/agent monitoring	3	gateway/cache, activity, agent health
Server operations	3	server registry, restart/deploy, status
Network/observability	3	SSH/Tailscale, logs, health polling
Provider/model usage	4	OAuth, usage probes, assignment, failover
Terminal/PC monitoring	2	terminal sessions, PC metrics
Total	53	

I would therefore think of this as **roughly 50 responsibilities**, not “server.py has three or four things going on.”

There are also several unmistakable compound functions. `recategorize_expense()`, for example, validates input, resolves taxonomy, locates DB rows, performs date-fallback matching, updates MySQL, updates static HTML, resolves report files, and records undo state. Pasted text(20260824-094016) +2 That's practically a miniature application in one function.

The architecture I would aim for

I think your “about 200 lines” instinct is right as a **guardrail**, but not as the seam itself. My rule would be:

One reason to change per module; normally 75–220 lines.

That probably leaves us with somewhere around **35–45 focused modules**, plus modules you've already extracted.

Something like this:

```
dashboard/
|
```

```
|— app.py                                # composition root only
|— server.py                             # temporary compatibility facade
|
|— contracts/
|   |— intake.py
|   |— finance.py
|   |— scanner.py
|   |— agents.py
|   |— operations.py
|   |— providers.py
|
|— ports/
|   |— persistence.py
|   |— processes.py
|   |— network.py
|   |— finance.py
|   |— agents.py
|
|— intake/
|   |— recent_repository.py
|   |— event_router.py
|   |— event_reducer.py
|   |— state_machine.py
|   |— dispatch_service.py
|   |— trainer_service.py
|   |— document_processor.py
|   |— statement_preflight.py
|
|— finance/
|   |— report_catalog.py
|   |— report_parser.py
|   |— report_resolver.py
|   |— report_renderer.py
|   |— expense_service.py
|   |— category_service.py
|   |— category_projection.py
|   |— vendor_review.py
|   |— receipt_index.py
|   |— supporting_documents.py
|
|— hardware/
|   |— scanner_registry.py
|   |— scanner_service.py
|   |— scanner_process.py
|   |— scanner_health.py
|   |— ...existing extracted modules...
|
|— agents/
|   |— registry.py
|   |— letta_adapter.py
|   |— model_preferences.py
|   |— voice_preferences.py
```

```

|   ├── activity.py
|   └── health.py
|
|   ├── operations/
|   │   ├── server_registry.py
|   │   ├── health_service.py
|   │   ├── restart_strategies.py
|   │   ├── ssh_service.py
|   │   └── log_service.py
|   |
|   ├── providers/
|   │   ├── usage_probe.py
|   │   ├── account_assignment.py
|   │   ├── failover.py
|   │   └── model_usage.py
|   |
|   └── http_api/
|       └── # already extracted

```

That is much closer to what I think this application actually is.

Where I would apply the GoF magic

This file has some **very good natural pattern seams**. I would not sprinkle patterns everywhere just to say we're using GoF; I'd hit the places where they eliminate branching and shared state.

Existing problem	Pattern	New object/seam
Scan/PDF/manual processing	Template Method + Strategy	<code>DocumentProcessor</code> + <code>DocumentSourceStrategy</code>
Mazda callbacks mutate giant intake dictionaries	Observer + State	<code>IntakeEventBus</code> , <code>IntakeStateMachine</code>
JSON pointer files	Repository	<code>IntakeRepository</code>
MySQL expenses	Repository	<code>ExpenseRepository</code>
scanner/WIA/AirScan/subprocesses	Adapter	<code>ScannerPort</code> implementations
source-document fallback logic	Chain of Responsibility	<code>SourceDocumentResolverChain</code>
receipt lookup fallbacks	Chain of Responsibility	<code>ReceiptResolverChain</code>
category update + undo	Command + Memento	<code>RecategorizeExpenseCommand</code>

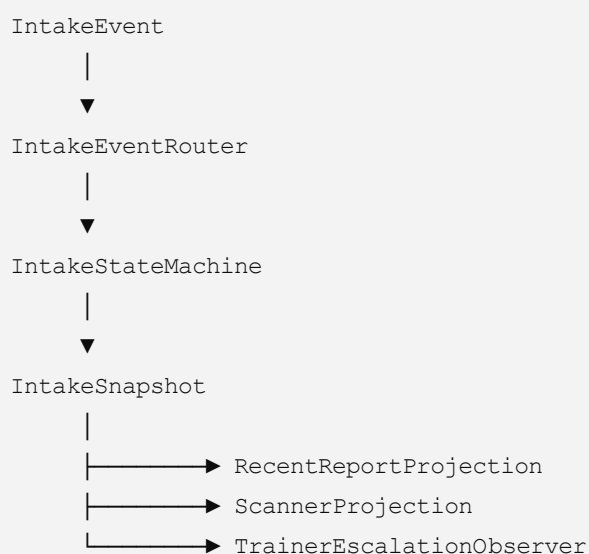
Existing problem	Pattern	New object/seam
static report recoloring	Observer / Projection	CategoryReportProjector
health checks	Strategy	HealthProbe
restart cases	Strategy	RestartStrategy
server definitions	Registry + Factory	ManagedServerRegistry
provider-specific usage calls	Strategy	UsageProbe
Letta/SSH/subprocess	Adapter	typed external ports
optional Trainer behavior	Null Object	you're already doing this

A particularly juicy one is the intake subsystem.

Right now the file is essentially implementing a state machine manually. It has terminal states, callback races, scanner correlation, transport failures, Trainer verdicts, human-only mode, and duplicate integrity behavior. Pasted text(20260824-094016) The comments even describe races between STEP 8 and Trainer updates. Pasted text(20260824-094016)

I would make that explicit.

Conceptually:



Then `_fold_event_into_intake()` disappears as a giant dictionary-mutating procedure.

Events could be discriminated Pydantic models such as:

```
IntakeDispatched
MazdaDispatchAccepted
MazdaDispatchFailed
ExpenseStored
StatementReviewed
TrainerVerdictReceived
HumanReviewRequired
VendorReviewRequired
ArchiveCompleted
IntakeFailed
```

That would make many of the strange race conditions **illegal state transitions** instead of **comments explaining why an `if` exists**.

Pydantic: yes, but not as the interfaces

One terminology tweak that will make the design cleaner:

Pydantic = contracts.

`Protocol` = interfaces.

The existing `ExecutionModeConfig` is actually a good example of Pydantic being used at the right seam: strict external configuration comes in and becomes a validated value.

Pasted text(20260824-094016)

I would probably end up with **20–30 Pydantic boundary models** and **15–25 Protocol interfaces**, rather than literally creating 100 Pydantic classes.

For example:

Pydantic contracts

```
ScanRequest
ScanResult
IntakeEvent
IntakeSnapshot
ExpenseStoredEvent
TrainerVerdict
ManualReceiptRequest
ExpenseEditRequest
RecategorizeRequest
SupportingDocumentQuery
AgentConfig
ServerStatus
ProviderUsage
HealthResult
```

Protocol ports

```
ScannerPort
IntakeRepository
ExpenseRepository
ReportRepository
MazdaDispatcher
TrainerNotifier
LettaGateway
ProcessRunner
SshClient
HealthProbe
RestartStrategy
CredentialStore
UsageProbe
Clock
```

That gives you the strict type boundary you're looking for without turning every private helper into a Pydantic ceremony.

The scanner is an especially clean extraction

The scanner section currently contains device definitions, WSL interop, printer repair integration, a global scan lock, stale-process cleanup, subprocess launching, timeout handling, result classification, and HTTP cache-busting.

Pasted text(20260824-094016) +2

I'd turn that into roughly:

ScannerRegistry	~100 lines
ScannerService	~150 lines
ScanProcessRunner	~150 lines
ScannerCoordinator	~100 lines
ScannerDiagnosticsService	~150 lines

`ScannerService` shouldn't know WSL exists.

It should basically say:

```
scanner = registry.get(key)
result = scanner.scan()
return result
```

Whether that's WIA through PowerShell, AirScan, a fake test scanner, or someday some Raspberry Pi network scanner becomes an Adapter/Strategy detail.

The nastiest seam: `process_scanned_document`

This one should become an **application use case**, not a grab-bag function.

Its current responsibility includes scan validation, quality checking, deterministic facade execution, statement preflight, staging, dispatch claiming, conversation creation, intake persistence, Mazda dispatch, and result shaping. The source itself describes the deterministic front end and agentic back end as distinct pipeline stages.

Pasted text(20260824-094016)

I'd make the outer service look conceptually like:

```
ProcessDocumentUseCase
|
├── DocumentValidator
├── DocumentClassifier
├── StatementPreflightStrategy
├── DocumentStager
└── DispatchClaimRepository
```

```
└─ ConversationFactory
└─ IntakeRepository
└─ MazdaDispatcher
```

Then `ProcessDocumentUseCase` can be 100–150 lines even though the total implementation behind it is considerably larger.

That is SRP done correctly: **the orchestrator owns the sequence, not all the work.**

There's also a Dependency Inversion problem at the bottom

The extraction of the HTTP handler was absolutely the right move, but it's currently transitional architecture. The comment says `http_app` reaches backward into the giant server module using `srv.<name> .` Pasted text(20260824-094016)

That makes `server.py` a kind of global **Service Locator**.

Eventually I want:

```
HTTP
|
▼
ApplicationServices
|
└─ FinanceService
└─ IntakeService
└─ ScannerService
└─ AgentService
└─ OperationsService
    |
    ▼
    Ports
    |
    ▼
    Adapters
```

Not:

```
http_app
|
▼
import server
|
▼
grab whatever global function happens to exist
```

So ultimately we'd construct something like an `AppServices` dependency container in `app.py` and inject it into `DashboardHandler` .

That is probably the final move, not the first one.

One more sign that configuration wants its own home

The agent configuration is mixed directly into the executable module with model configuration, voice configuration, caches, providers, etc.

Pasted text(20260824-094016) +1

And there's already configuration drift: **Shelia appears twice in** `LETTA_AGENTS`.

Pasted text(20260824-094016)

That's exactly the sort of thing a strict:

```
AgentRegistry
List[AgentDefinition]
```

Pydantic model can reject at startup with a uniqueness validator.

No more silent duplicate registry entries.

How I would actually attack this without breaking Mazda

I would use a **Strangler / Branch-by-Abstraction refactor** rather than rewriting

`server.py`.

1. **Characterization tests first.** Freeze HTTP responses, important filesystem effects, DB mutations, callback behavior, scan behavior, and Trainer/Mazda correlation. We specifically preserve the weird behavior before trying to make it pretty.
2. **Introduce contracts and ports without moving behavior.** Pydantic request/event/result models plus Protocols for repositories, subprocess, Letta, scanners, SSH, clocks, etc.
3. **Extract recent-intake state first.** It has some of the highest concurrency/race complexity. Create `IntakeRepository`, `IntakeEventReducer`, and `IntakeStateMachine`. Keep old `server.merge_recent_intake_event()` as a compatibility forwarding function.
4. **Break** `recategorize_expense()` **next.** Extract `ExpenseMatcher`, `RecategorizeExpenseHandler`, `CategoryReportProjector`, and the already-existing undo service. The HTTP behavior doesn't change.
5. **Extract scanner orchestration.** Keep `server.run_scanner()` as a one-line compatibility facade while tests continue monkeypatching the historical names.
6. **Extract** `ProcessDocumentUseCase`. Scan and PDF become strategies feeding one shared pipeline rather than parallel procedural paths.
7. **Move report/receipt/supporting-document logic.** These have excellent seams because much of the work is already pure path resolution, matching, and rendering.

8. **Move agent/provider/server-management subsystems.** Registry, Letta adapter, health probes, OAuth usage, failover, restart strategies, SSH and logs become independent packages.
9. **Invert** `http_app` . Inject application services instead of letting routes dynamically reach into `server` .
10. **Kill the compatibility facade last.** `server.py` ends up at roughly **100–200 lines** and does little more than assemble dependencies and call `serve()` .

The last few lines of the current file already resemble the destination: construct background tasks and call `serve()` . Pasted text(20260824-094016) We basically want **the entire file to look like those last 70 lines.**

The key migration trick

There's something I particularly like in your existing code: extracted modules are deliberately **re-exported under their historical** `server` **names so old callers and tests continue to work.** The scanner comments explicitly describe doing this.

Pasted text(20260824-094016)

I would use that trick relentlessly:

```
Before:
    server.py owns implementation

Step 1:
    new_module.py owns implementation
    server.py imports/re-exports old name

Step 2:
    tests pass unchanged

Step 3:
    callers migrate to interface/service

Step 4:
    compatibility export disappears
```

That lets us dismantle this monster **one seam at a time with no big-bang rewrite.**

So my target would **not** be “take 8,651 lines and mechanically create forty-three 200-line files.” My target would be:

Turn `server.py` into a composition root, create **~35–45 cohesive modules, ~20–30 strict Pydantic contracts, ~15–25 Protocol ports, and use GoF patterns exactly where they eliminate branching, state mutation, or concrete dependencies.**

And I would start with **Recent Intake/Event State** → **Recategorization** → **Scanner** → **Process Document**. Those four cuts will probably reveal the architecture for the rest of the system.

I also surfaced the **Superpowers** plugin for you. If you connect it, it can be useful when we move from this architecture plan into the actual step-by-step extraction/refactoring work.

Sources